

AIDA vs ECC

A verified competitive analysis — Everything Claude Code (`affaan-m/ecc`, v2.0.0-rc.1)
against AIDA’s git-canonical project-truth substrate

Produced via a multi-agent adversarial review loop (Claude + Codex). See Methodology.

2026-06-05

Contents

Executive summary	1
The two tools, in one line each	2
ECC: four layers, only one of which contests AIDA	2
AIDA: the substrate ECC doesn’t have	2
Head-to-head	4
Verified facts and the caveats that matter	4
What AIDA should take from ECC	5
Verdict	5
Methodology	6

Executive summary

ECC and AIDA look adjacent but are different categories. ECC (Everything Claude Code) is a **cross-harness agent operating pack and distribution platform** — a large prompt/skill catalog, a mature installer, a real hook-enforcement layer, and a newly-added local Rust control plane (`ecc2`). AIDA is a **git-canonical project-truth substrate** — a durable spec graph with stable IDs, typed relationships, code-to-spec traces, a requirement lifecycle, and an MCP server, designed to survive every conversation and every vendor switch.

They contest exactly **one** surface: agent-orchestration runtime (ECC’s `ecc2` session daemon vs AIDA’s lease/queue/drain). They diverge completely on **durable, ID-stable, traceable, vendor-neutral truth** — which ECC structurally lacks and which is AIDA’s entire moat.

The one action AIDA should take now: a CI guard that fails the build when AIDA’s own template system drifts (the strongest idea borrowed from ECC’s installer). Everything else ECC does well is distribution/polish that AIDA should defer until its stability phase clears.

The two tools, in one line each

- **ECC** — *expertise and tooling, packaged for every harness*. ~251 skills, 63 agents, 115 rules, 47 enforcement hooks, an 11-target installer, and an alpha Rust TUI/daemon, distributed via npm and three plugin marketplaces. GitHub reports **207,592 stars** (see caveat below).
- **AIDA** — *the project's durable memory and rails*. An orphan-branch YAML spec graph (one file per requirement), stable SPEC-IDs, typed relationships, `// trace:` code links, a Draft→Completed→Released lifecycle, leases, a queue/orchestrator drain, briefs, and an MCP server — all git-canonical, with SQLite used only as a rebuildable cache.

ECC: four layers, only one of which contests AIDA

ECC is not one system. It is four, stacked:

Layer 1 — Prompt/process marketplace (the mass). 251 skills, 63 agents, 79 commands, 115 rules. By byte volume it is ~93% **Markdown vs code**, and **zero skills reference an ecc engine** — it is expertise-as-prompts, substrate-*unaware*. Its own catalog counts disagree across metadata (SOUL.md says 135; filesystem says 251), so a `catalog:check` script exists to fight the drift.

Layer 2 — Distribution engine (genuinely strong, ~7.5/10). A manifest-driven installer (modules → profiles → components), JSON-schema-validated, with a real plan/apply dry-run, an install-state record, and clean uninstall. Its standout is a **CI validator that fails the build if any declared path is missing or two modules claim the same path** — and an “honesty matrix” grading each harness Native / Adapter / Instruction / Reference.

Layer 3 — Hook enforcement floor (~9K LOC, 47 hooks). Format/typecheck gates, `--no-verify` blocking, MCP-health that survives compaction, session persistence. Production-grade — but generic dev-hygiene, decoupled from any requirement state.

Layer 4 — ecc2, a working alpha control plane (the contested surface). Real Rust (52,141 lines across 16 files, 425 test markers, builds with 24 warnings), a `ratatui` TUI, `git2` worktrees, session/delegation daemon, cron, OTEL spans, and a **token-authenticated HTTP intake endpoint** for remote dispatch. But its truth lives in `~/.claude/ecc2.db` — **local SQLite, single-machine, the source of record (not a rebuildable cache)**. No git-canonical store, no stable requirement IDs, no typed requirement graph, no code-to-spec traces, no merge-anchored lifecycle.

AIDA: the substrate ECC doesn't have

AIDA's defensible layer is the part that *doesn't* end with a conversation:

- **Git-canonical truth** — requirements are YAML on the orphan `aida-store` branch; SQLite is a disposable cache. Wipe the cache and it rebuilds from git. It survives machine resets, clones, and vendor switches.
- **Stable IDs + typed relationships** — TASK-N, BUG-N, STORY-N survive renames, merges, and releases; `blocks` / `blocked-by` edges are first-class.
- **Code-to-spec traceability** — `// trace:SPEC-ID` comments plus commit-trailer auto-bump tie source to requirements and promote `done` → `completed` on merge.

- **A requirement lifecycle** — Draft → Approved → Planned → In Progress → Done → Completed → Released, anchored to git history, not free-string Kanban lanes.
- **Multi-agent governance** — leases, briefs, sketch-first sign-off, PR-anchored orchestration phases.
- **Vendor neutrality** — the same graph is queryable over MCP by any agent (Claude, Codex, Antigravity), not just one harness.

Head-to-head

Vector	ECC	AIDA
Category	Cross-harness operating pack + marketplace + alpha control plane	Project-truth substrate with an orchestrator on top
Source of truth	~/ .claude/ecc2.db (local SQLite, single-machine)	Orphan-branch YAML in git; SQLite is a rebuildable cache
Identity	Ephemeral session UUIDs	Stable spec IDs surviving rename / merge / release / vendor
Traceability	None (free-text decision logs)	// trace:SPEC-ID + commit-trailer auto-bump
Lifecycle	Process states + free-string Kanban lanes	Merge-anchored Draft → Completed → Released
Distribution	Manifest installer, 11 harnesses, npm, 3 marketplaces	aida init + symlink dogfooding; docs, no manifest
Orchestration	Session/worktree daemon, auto-merge, cron, OTEL, TUI	Leases, queue, --auto-complete drain, PR-anchored phases, TUI
Cross-machine / team	No (local DB)	Yes (git-canonical, syncable)

Where they compete: the orchestration runtime only. ECC’s `ecc2` has a richer operator *dashboard* today (OTEL spans, risk scoring, conflict incidents, cron). AIDA’s drain is spec-grounded and PR-anchored.

Where they complement / diverge: everything else. ECC is strong on *getting expertise and tooling onto many harnesses*; AIDA is strong on *being the durable, vendor-neutral record of what the project is*. ECC has no analog to AIDA’s spec graph; AIDA has no analog to ECC’s manifest installer.

Verified facts and the caveats that matter

This report’s facts were independently reproduced (see Methodology). Three caveats are load-bearing:

1. **“207k stars” is GitHub-reported, not mindshare-proven.** The live API confirms 207,592 stars / 31,858 forks for `affaan-m/ecc` (created 2026-01-18; `everything-claude-code` resolves to the same object). Deep stargazer pagination is blocked, so *organic adoption was not verifiable*. Cite it as “GitHub reports 207k stars,” never as “207k users.” It is nonetheless a real **distribution-reach** signal AIDA lacks.
2. **`ecc2` is more capable than a first pass suggests — but still local-truth-bound.** Early analysis called its remote dispatch “a misnomer” and its draft-PR path “unwired”; reproduction against the source corrected both. ECC has a **real network intake** (token-auth HTTP POST `/dispatch`) with **local execution**, and a **real TUI draft-PR path** (the dashboard calls `create_draft_pr_with_options`). The accurate framing: ECC has

genuine operator-workflow machinery that *terminates in a local, SQLite-backed control plane* — competitive on workflow, still absent the canonical-truth layer.

3. **The asymmetry favors AIDA, but qualitatively, not absolutely.** AIDA can close its distribution gap (a manifest + installer) in a bounded effort; ECC closing the *truth* gap means re-architecting onto a git-canonical, ID-stable, traceable substrate — deeper and against its ship-fast / local-SQLite grain. But “ECC cannot cross this gap” is rhetoric: a thin git-export/sync layer over its existing SQLite could become “good-enough shared truth” for many users. **AIDA’s moat is real; it is not a moat AIDA can stop defending.**

What AIDA should take from ECC

The catalog and the breadth are **not** the lesson — ECC’s 251-skill sprawl, count-rot, and forked-installer maintenance debt are exactly the surface complexity AIDA deliberately avoids. The lesson is the *distribution discipline*, and only one slice of it is on-phase right now:

- **Do now — the template-drift CI guard.** AIDA’s dual-copy template system (`build.rs`-embedded `aida-core/templates/` + per-file `.claude/` symlinks) is kept in sync today only by `make sync-templates` and manual discipline. Borrow ECC’s validator idea as a **cargo test** that fails the build if any embedded-template path is missing or two scaffold sources claim the same destination. **This is not “borrow ECC’s installer” — it is a testable guard against AIDA’s own known drift risk**, which makes it stability work, not marketing.
- **Defer (backlog) — install-state + uninstall, manifest profiles, ecc2-style TUI telemetry.** Genuine ideas, but distribution/polish that should wait until AIDA’s bug backlog clears.
- **Cheap and on-phase — an honesty-graded harness matrix, documentation only.** Grade AIDA’s per-harness support (Native / Adapter / Instruction) in the positioning docs so claims never outrun enforcement. Keep it as docs; do not let it become adapter work.

Verdict

AIDA and ECC are not substitutes. ECC is the better answer to “*get expertise and tooling onto every coding harness.*” AIDA is the better answer to “*what is the durable, vendor-neutral, queryable record of this project, and is the code still tied to a live requirement?*” The orchestration overlap is real and worth watching — ECC is building *down* from a marketplace into orchestration at notable velocity (v1.9 → v2.0 in roughly 2.5 months) — but ECC’s control plane is a local session tracker without a truth layer, and that is the one thing hardest for it to add and easiest for AIDA to defend.

The disciplined move is not to chase ECC’s breadth. It is to take the single template-drift guard now, bank the rest as backlog, and keep the truth-moat legible.

Methodology

This report is the reconciled output of a deliberate **multi-agent adversarial loop**, not a single pass — which is itself a demonstration of the rigor AIDA’s substrate is built to support:

1. **Three independent analyses** of `affaan-m/ecc` were produced: a README-only verdict, a deep dive (full clone + three parallel subagents), and a committed synthesis document.
2. **A critique** checked the third against the live repository and AIDA store.
3. **A consolidated, brutally-honest review** graded all three analyses *and the critique* against each other, surfacing that every prior artifact — including the critique — had carried a wrong star figure because none had queried the live source.
4. **An independent reviewer (Codex)** was given the consolidated review with explicit instructions to *reproduce, not agree*. It built `ecc2`, re-queried the API, and corrected two overstated claims (remote dispatch, draft-PR wiring) with file-and-line evidence.
5. **Reconciliation** (this document) verified Codex’s corrections independently and folded them in.

The exercise produced one durable lesson worth stating plainly: **a delegated finding is not a verified fact**. A synthesis is only as sound as its weakest un-reproduced input, and the corrections here landed exactly in the zone that had been delegated and not personally re-run. Forcing reproduction — the loop’s core mechanic — is what caught them.

Full working artifacts, including the internal review and the Codex rebuttal, are retained under docs/competitive-analysis/ and anchored to SPIKE-50.